

HelixQA

HelixQA

Анти-блуфф оркестрација QA теста — аутономне, просс-платформ сесије у којима свако УСПЕШНО извршење носи прикупљене доказе да стварни корисник може да користи функционалност.

Сажетак

HelixQA је анти-блуфф оквир за оркестрацију QA теста на више платформи (Android, Android TV, Web, Десктоп) који комбинује YAML банке тестова, детекцију пада у реалном времену, прикупљање доказа корак по корак и LLM-плус-рачунарски-видом вођене аутономне QA сесије како би доказао да функционалности заиста раде од почетка до краја. Представља обавезни тип QA теста према Constitution (§11.4.169).

Кратак опис

Анти-блуфф QA оркестратор (Go) који покреће написане банке тестова и потпуно аутономне, LLM-и-рачунарским-видом вођене QA сесије на различитим платформама — открива падове, валидира сваки корак у односу на прикупљене доказе (снимке екрана, логцат, видео, стацк траце), и аутоматски генерише тикете богате доказима за AI пипелине поправки.

Детаљан опис

HelixQA је Go оквир чији једини, непопустљиви дизајнерски центар представља §11.4 Оперативно правило Constitution: критеријум за испоруку није „тестови пролазе“, већ „корисници могу да користе функционалност“, па свако УСПЕШНО извршење које емитује мора да носи позитивне доказе прикупљене током извршавања — без доказа, нема зеленог светла, без изузетака. Ради у два комплементарна мода који заједно покривају и писане и непознате сценарије.

Прво, **писане банке тестова** — YAML пакети TC-XXX случајева са циљањем на платформу, приоритетима, уређеним корацима (назив/акција/очекивано), ознакама и референцама на документацију — извршавају се уз валидацију сваког корака, детекцију пада/ANR-а у реалном времену (ADB за Android, праћење процеса за web/десктоп), централизовано прикупљање доказа и аутоматски генерисане Markdown тикете већ прилагођене за downstream AI пипелине поправки. Друго, **потпуно аутономна QA сесија** која апликацију предаје агентима покретаним LLM и рачунарским видом и пушта их да је самостално контролишу кроз четири дисциплиноване фазе: подешавање (избор LLM-ова, израда мапе функционалности из пројектног документа, покретање CLI агената, иницијализација мотора за визуелну анализу), верификација вођена документацијом која пролази кроз сваку документовану функционалност, истраживање вођено радозналешћу које намерно тестира граничне случајеве и недокументовано понашање, а затим извештавање и чишћење у Markdown/HTML/JSON формату, при чему је сваки налаз повезан са доказима означеним временским печатом у видео-запису.

Кључно је да овај оквир не оцењује сам себе: интегрише четири спољна Go потмодула (LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor) и користи заједничку инфраструктуру challenges и containers, тако да компонента која навиђа апликацијом није иста као она која оцењује да ли је све прошло како треба. Његов сопствени пакет тестова подвргава се истом стандарду који намеће другима путем make anti-bluff (статичко скенирање + манифест сидрених понашања + мутациони зупчаник) и Цхалленге оркестрације у 8 фаза са уграђеном §1.1 мутацијом. Матрица покривености типова тестова од 15 редова прецизно повезује сваку рекламирану могућност са конкретним извршним средством и специфичним обликом прикупљених доказа — тако да тврдње оквира о самом себи имају исто толико доказа колико и пресуде које доноси за производе које тестира.

Зашто смо га изградили

Конвенционални QA даје зелено светло уз образложење „асертација је прошла“, што је управо начин на који класа грешака коју Constitution назива *блефирањем* пролази кроз систем — функција је пријављена као функционална, иако је за крајњег корисника неисправна. HelixQA је изграђен управо да то спречи у оквиру QA процеса: он одбија да додели статус ПРОШАО без физичког доказа (снимка екрана, логцат, видео, стек трејс, извештај) снимљеног током стварног извршавања, а зелени резиме без таквог доказа третира као критичну грешку

равну недостајућој функцији. Такође решава проблем радне снаге — свеобухватно ручно тестирање на више платформи није скалабилно — омогућавајући потпуно аутономне сесије.

Зашто је револуционарно

Спаја две ствари које готово никада нису део истог алата: ригорозну, доказима поткрепљену контролу квалитета и аутономно, самостално истраживање. Агент LLM са визуелним модулom отвара *стварну* апликацију, проверава сваку документовану функцију, трага за недокументованим баговима за које нико није написао тест, *и* истовремено генерише доказни материјал судског квалитета — тако да се „тестирали смо то“ замењује са „ево снимка, ево логцат-а, ево тикета“. А пошто је део QA подсистема названог по Constitution, његово усвајање не унапређује поштење QA процеса само за један тим — већ подиже стандард за сваки производ у породици у једном потезу.

Шта је иновативно

- **Уговор о недоказивом блефу** — сваки ПРОШАО статус је везан за снимљени доказ током извршавања; зелена линија у ЦИ-у сматра се неопходном, али никада довољном, а зелени резиме без доказа оцењује се као критична грешка.
- **Аутономно истраживање вођено документацијом и радозналостју** — проверава сваку документовану функцију *и* затим скреће са сценарија, истражујући рубне случајеве на које наилазе стварни корисници (празни уноси, брзе интеракције, недокументовани путеви) које ниједан ручно написан сет тестова није предвидео.
- **Визуелни оракул** — механички вид заснован на GoCV-у и LLM Висион API буквално *види* покренути UI на екрану, откривајући визуелно оштећена стања која промичу поред асертација на нивоу токена и својстава.
- **Структурне, а не прозне базе тестова** — стрингови у бази описују структуру и генеришу питања за LLM током извршавања (ЦОНСТ-046), тако да једна база функционише на свим локализацијама уместо да се распадне чим се UI текст преведе.
- **Тикети прилагођени AI протоку поправки** — аутоматски генерисани Markdown тикети стижу са комплетним пакетом доказа, спремни за директно прослеђивање агенту за поправке уместо људском тријажеру.

Како се користи у свим производима (моћи које пружа)

Као **обавезни стуб квалитета** (Constitution §11.4.169 наводи helix_qa подсистем као један од обавезних типова тестова), HelixQA свим производима у породици пружа исти скуп могућности:

- **Аутономне QA сесије:** једна наредба `helixqa autonomous --project ... --platforms android,desktop,web` пушта агента LLM са визуелним модулом да самостално тестира стварне апликације према циљу покривености, генеришући извештаје, тикете и снимке без људске интервенције.
- **Базе тестова / сетови:** базе YAML (ниво 219 са минимално 30 тестова), циљане на платформе, рангиране по приоритету и повезане ред по ред са документацијом коју проверавају.
- **Снимљени докази:** снимци екрана, логцат, видео, стек трејсови и комплетна временска линија — централизовани и повезани са сваким извештајем, тако да се свака одлука може репродуковати и ревидирати накнадно.
- **Независне пресуде (§11.4.141 принцип независности):** његов `issuedetector` и визуелни оракул на бази LLM оцењују понашање покренуте апликације независно од агента који ју је навигирао, чиме се структурно елиминише класична грешка у којој систем сам себе проглашава исправним.
- **Контрола и мутациони механизам:** `make qa-all / make anti-bluff` и `challenges/scripts/helixqa_orchestrator_challenge.sh` (8 фаза, уграђена §1.1 мутација) непрестано проверавају поштење самог HelixQA — и намерно не постоји `--skip-helixqa` пречица којом би се дисциплина могла искључити под притиском рокова.

Највећи технички изазови и како смо их решили

- **Спречавање лажно позитивних резултата у самом QA процесу** — алат који открива блефове не сме сам постати један → сваки корак се валидира на основу прикупљених доказа, пролаз без доказа оцењује се као грешка, а не као пролаз, а манифест понашајних сидришта повезује сваку рекламирану функционалност са извршним тестом (ЦОНСТ-035), тако да се ниједна функција не може тврдити без нечега што је проверава.
- **Управљање хетерогеним платформама из једног центра** — Android, Android TV, Web и Десктоп немају заједнички модел уноса → један пакет `navigator` апстрахује платформски специфичне `ActionExecutors` (ADB, Playwright, X11) и детекторе падова за сваку платформу (андرويد/web/

десктоп), тако да се логика оркестрације пише једном, а разлике између платформи остају на ивицама система.

- **Коришћење аутономних агената на користан, а не хаотичан начин** — нерегулисани LLM у апликацији може бесциљно лутати вечито → LLMsVerifier оцењује и бира одговарајуће моделе, LLMOrchestrator управља хеадлесс CLI агентима (опенцоде, цлауде-цоде, гемини, јуние, qwen-цоде), DocProcessor гради мапу функционалности која истраживању даје циљ, а VisionEngine сваку одлуку заснива на стварним пикселима на екрану, а не на машини модела.
- **Базе тестова безбедне за локализацију** — пакет који хардкодира енглески текст корисничког интерфејса отказује у петнаест језика → базе описују само структуру, а текст корисничког упита се учитава динамички путем LLM/ресурса током извршавања (ЦОНСТ-046), тако да иста база проверава исто понашање без обзира на локализацију.
- **Доказивање да контролни механизми нису лажни** — анти-блеф контрола која сама не може да откаже представља врхунски блеф → упарени §1.1 мутанти уклањају прикупљање доказа или анти-блеф тврдњу из типа и захтевају да контрола ОТКАЖЕ, а механизам за постепено повећање захтева спречава да се та гаранција временом неприметно наруши.

Технолошки стек

- **Go 1.24+ оркестратор** — *зашто*: QA мора да ради свуда где раде и производи, па један статички повезан, брз и преносив бинарни фајл побеђује алтернативе које захтевају рунтине; *како*: један cmd/helixqa CLI који излаже композитне поткоманде run / list / report / autonomous / version.
- **Базе тестова YAML (pkg/testbank)** — *зашто*: пакети тестова треба да буду декларативни и читљиви, мењиви од стране људи без додиривања Go; *како*: version/name/test_cases[] са id, category, priority, platforms, уређеним низом steps[] и documentation_refs[] за повратно праћење до документације функционалности.
- **Детектори падова/ANR-а (pkg/detector)** — *зашто*: најважније грешке су оне које се дешавају уживо, током интеракције, а не у накнадној провери; *како*: ADB (pidof/logcat/screenshot) за Android и pgrep за web/десктоп, који прате процес док га тест покреће.
- **Прикупљање доказа (pkg/evidence, pkg/session)** — *зашто*: уговор о анти-блефу је стваран само ако је сваки ПРОЛАЗ подржан физичким доказом; *како*: снимци екрана, логат, видео и стацк траце-ови се прикупљају у временску линију SessionRecorder, на коју сваки извештај упућује.
- **Аутономна сесија (pkg/autonomous, pkg/navigator, pkg/issuedetector)** — *зашто*: свеобухватно ручно QA тестирање

на четири платформе не може да се скалира, па истраживање мора да буде самостално; *како*: 4-фазни SessionCoordinator плус ActionExecutors (ADB/Playwright/X11) и LLM детекција грешака која обухвата визуелне, UX, приступачне и функционалне недостатке.

- **Спољашњи потмодули** — *зашто*: поновна употреба и раздвајање (ЦОНСТ-051), а — кључно — раздвајање навигатора од суца; *како*: LLMsVerifier (оцењивање модела), LLMOrchestrator (хеадлесс CLI агенти), VisionEngine (GoCV + LLM Висион), DocProcessor (мапа функционалности/покривеност), сваки као независно вођена компонента.
- **Анти-блеф контроле + механизам постепеног повећања захтева** — *зашто*: да се HelixQA држи тачно оног §1.1 споразума који намеће свему осталом; *како*: скенирање make anti-bluff плус манифест понашајних сидришта и механизам постепеног повећања захтева, са helixqa_orchestrator_challenge.sh као 8-фазним валидатором од краја до краја.
- **Матрица покривености од 15 редова (docs/test-coverage.md)** — *зашто*: ЦОНСТ-050(Б) захтева затворен, потпуно обрачунат скуп типова тестова без празнина; *како*: сваки ред је везан за конкретан извршни ресурс и специфичан облик прикупљених доказа, тако да је покривеност проверена чињеница, а не само тврдња.

Напомене о статусу и искрености

- **Статус: бета.** Активно у развоју (РЕАДМЕ статусна трака, верзија 219). Подвргава се сопственом стандарду против обмане.
- **Лиценца: Апацхе-2.0.** Инсталација: `go install digital.vasic.helixqa/cmd/helixqa@latest`.

Приоритетни ниво: Helix-основни — обавезни стуб квалитета/против обмане у оквиру породице Helix, којим се проверава да функције заиста раде.